*apply family

The problem

We have the reaction times (in ms) of three participants, saved in a list:

```r
rts <- list(anna = c(450, 520, 380),
            ben  = c(610, 580),
            carl = c(400, 430, 470, 500))
rts
```

```
$anna
[1] 450 520 380

$ben
[1] 610 580

$carl
[1] 400 430 470 500
```

We want the **mean of each participant**: apply the **same function** (`mean`) to **each element** of the list.

Solution 1: a `for` loop

```
means <- rep(NA, length(rts))   # empty vector to fill

for (i in 1:length(rts)) {
  means[i] <- mean(rts[[i]])    # mean of the i-th element
}
means
```

```
[1] 450 595 450
```

It works, but:

- we need to create the empty vector first
- we need to handle the index `i`, both for `rts` and for `means`

Solution 2: `sapply()`

`sapply(rts, mean)`: apply `mean` to **each element** of `rts`

```
1 sapply(rts, mean)
```

```
anna  ben carl
 450  595  450
```

It is exactly the same as writing, one by one:

```
1 c(anna = mean(rts$anna),
2   ben  = mean(rts$ben),
3   carl = mean(rts$carl))
```

```
anna  ben carl
 450  595  450
```

… but `sapply()` does it for us, for any number of elements, and keeps the names.

The structure

`sapply(X, FUN)`

- `X`: the object we go through, **one element at a time**
  - a list (each element), a vector (each value), a dataframe (each column)
- `FUN`: the function applied to **each element**
  - a function of R or of a package (`mean`, `length`, …)
  - a function we created
  - an anonymous function, written on the spot

Like a `for` loop

`sapply()` is a compact `for` loop: the element of `X` plays the role of `i`

`for` loop

```
for (x in 1:3) {
  print(x * 10)
}
```

```
[1] 10
[1] 20
[1] 30
```

`sapply`

```
sapply(1:3, function(x) x * 10)
```

```
[1] 10 20 30
```

Both take **1, then 2, then 3** and multiply each by 10. `sapply()` collects the results in a vector.

`lapply()` vs `sapply()`

Same input (`rts`), same function (`length`: how many trials each participant has)

`lapply`: always returns a **list**

```
1 lapply(rts, length)
```

```
$anna
[1] 3

$ben
[1] 2

$carl
[1] 4
```

`sapply`: **simplifies** the result, here to a vector

```
1 sapply(rts, length)
```

```
anna  ben carl
   3    2    4
```

With our own function

A function that converts milliseconds into seconds…

```
1 to_seconds <- function(x) x / 1000
2 to_seconds(rts$anna)   # on one element
```

```
[1] 0.45 0.52 0.38
```

… and apply it to **each element** of `rts`:

```
1 lapply(rts, to_seconds)
```

```
$anna
[1] 0.45 0.52 0.38

$ben
[1] 0.61 0.58

$carl
[1] 0.40 0.43 0.47 0.50
```

With an anonymous function

If we need the function only once, we can write it **directly inside** `sapply()`, without saving it:

```
# range of each participant: maximum - minimum
sapply(rts, function(x) max(x) - min(x))
```

```
anna  ben carl
 140   30  100
```

`x` is a **placeholder** (like `i` in a `for` loop): it becomes each element of `rts`, one at a time

- first `x` is `rts$anna`: `520 - 380 = 140`
- then `x` is `rts$ben`: `610 - 580 = 30`
- then `x` is `rts$carl`: `500 - 400 = 100`

All the types of `*apply`

function	goes through …	returns
`lapply()`	each element of a list/vector	a list
`sapply()`	each element of a list/vector	a vector or matrix, if possible
`apply()`	each row or column of a matrix/dataframe	a vector
`tapply()`	each group of a vector, defined by a factor	a vector
`vapply()`	like `sapply()`, but checks the type of the result	a vector
`mapply()`	several vectors/lists at the same time	a vector or a list

`apply()`: rows or columns

The scores of 3 students in 3 tests: one **row** for each student, one **column** for each test

```r
scores <- data.frame(test1 = c(24, 18, 30),
                     test2 = c(28, 22, 29),
                     test3 = c(30, 25, 27))
scores
```

```
  test1 test2 test3
1    24    28    30
2    18    22    25
3    30    29    27
```

`apply(X, MARGIN, FUN)`: `MARGIN = 1` for the **rows**, `MARGIN = 2` for the **columns**

`apply()`: rows or columns

`MARGIN = 1`: `mean` of **each row**, one result per **student** (first, second, third student)

```
1 apply(scores, MARGIN = 1, FUN = mean)
```

```
[1] 27.33333 21.66667 28.66667
```

`MARGIN = 2`: `max` of **each column**, one result per **test** (first, second, third test)

```
1 apply(scores, MARGIN = 2, FUN = max)
```

```
test1 test2 test3
   30    29    30
```

Another example: the mean of each numeric column of `iris`

```
1 apply(iris[, 1:4], MARGIN = 2, FUN = mean)
```

```
Sepal.Length  Sepal.Width Petal.Length  Petal.Width
    5.843333     3.057333     3.758000     1.199333
```

`tapply()`: by group

Reaction times of 6 trials, in two conditions:

```r
exp_data <- data.frame(rt = c(450, 520, 380, 610, 580, 640),
  condition = rep(c("easy", "hard"), each = 3))
exp_data
```

```
  rt condition
1 450      easy
2 520      easy
3 380      easy
4 610      hard
5 580      hard
6 640      hard
```

`tapply(X, INDEX, FUN)`: apply **`FUN`** to **`X`**, group by group (**`INDEX`**)

```r
tapply(exp_data$rt, INDEX = exp_data$condition, FUN = mean)
```

```
easy hard
 450  610
```

`tapply()`: by group

It is the same as selecting each group by hand:

```
1 mean(exp_data$rt[exp_data$condition == "easy"])
```

```
[1] 450
```

```
1 mean(exp_data$rt[exp_data$condition == "hard"])
```

```
[1] 610
```

With `tapply()` we write it once, whatever the number of groups. For example, the mean `Petal.Length` of each species in `iris`:

```
1 tapply(iris$Petal.Length, INDEX = iris$Species, FUN = mean)
```

```
    setosa versicolor  virginica 
     1.462      4.260      5.552 
```

`vapply()`: checking the result

Like `sapply()`, but we declare **in advance** what each result must be (`FUN.VALUE`): here, **one number**

```
1 vapply(rts, FUN = mean, FUN.VALUE = numeric(1))
```

```
anna  ben carl
 450  595  450
```

If the function returns something different, `vapply()` stops with an error. `range()` returns **two** numbers:

```
1 vapply(rts, FUN = range, FUN.VALUE = numeric(1))
```

```
Error in `vapply()`:
! values must be length 1,
 but FUN(X[[1]]) result is length 2
```

Useful inside your own functions, to catch mistakes early.

`mapply()`: two vectors at the same time

`sapply()` goes through **one** vector, `mapply()` goes through **two** (or more) vectors **together**

```r
x <- c(1, 2, 3)
y <- c(10, 20, 30)
mapply(function(a, b) a + b, x, y)
```

```
[1] 11 22 33
```

step	`a` (from `x`)	`b` (from `y`)	`a + b`
1	1	10	11
2	2	20	22
3	3	30	33

`x` and `y` must have the **same length**

`mapply()` with `rep()`

Repeat each value a **different** number of times

```
1 values <- c(5, 10, 15)
2 n_rep <- c(3, 2, 1)
3 mapply(rep, values, n_rep)
```

```
[[1]]
[1] 5 5 5

[[2]]
[1] 10 10

[[3]]
[1] 15
```

Step by step, `mapply()` runs:

```
1 rep(5, 3)      # step 1
```

```
[1] 5 5 5
```

```
1 rep(10, 2)     # step 2
```

```
[1] 10 10
```

```
1 rep(15, 1)     # step 3
```

```
[1] 15
```

`mapply()`: simulating data

Three samples of 3 values, each with its own mean and standard deviation:

```
1 set.seed(1)
2 mus <- c(10, 50, 100)
3 sigmas <- c(1, 5, 10)
4 mapply(function(m, s) rnorm(3, mean = m, sd = s), mus, sigmas)
```

```
         [,1]     [,2]     [,3]
[1,]  9.373546 57.97640 104.8743
[2,] 10.183643 51.64754 107.3832
[3,]  9.164371 45.89766 105.7578
```

- one **column** per call: `m = 10, s = 1`, then `m = 50, s = 5`, then `m = 100, s = 10`
- add `SIMPLIFY = FALSE` to get a list instead of a matrix

`replicate()`: repeat the same code

`replicate(n, expr)`: run the code `expr` `n` times. There is no element to go through: the code is just repeated.

```
1 set.seed(1)
2 replicate(n = 5, expr = mean(rnorm(30, mean = 100, sd = 30)))
```

```
[1] 102.47375 103.98324 103.30834 103.39998  90.09916
```

Each number is the mean of a **new** sample of 30 values: the means are all different, but close to 100.

`replicate()`: sampling distribution

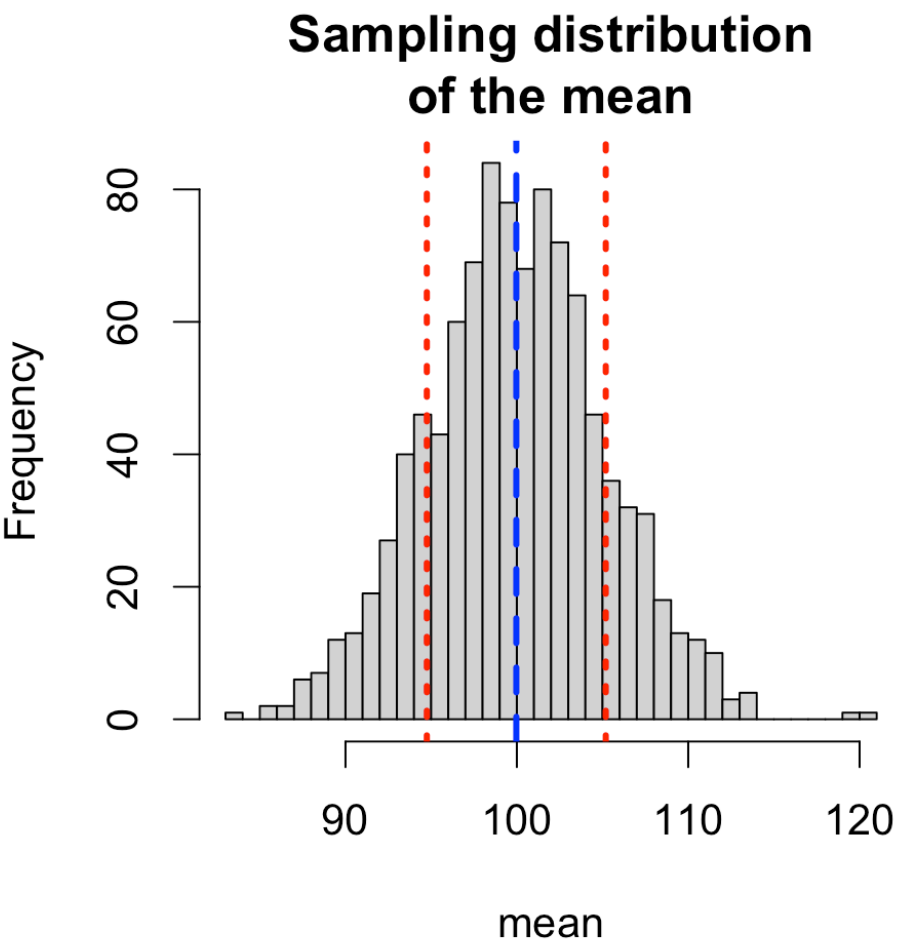
Repeating it 1000 times we get the **sampling distribution of the mean**:

```r
set.seed(1)
means <- replicate(
  n = 1000,
  expr = {
    x <- rnorm(30, 100, 30)
    mean(x)
  })
mean(means)
```

```
[1] 99.96828
```

```r
sd(means)  # standard error
```

```
[1] 5.220686
```



Now let’s practice!

Open this link and keep it open:

https://etherpad.wikimedia.org/p/arca-corsoR